

# Binary Sentiment Classification Using KNIME

Luciana O. Dias

April 2026

## 1 Pipeline Description

The workflow implements a complete binary sentiment classification pipeline using KNIME, covering data ingestion, text preprocessing, feature engineering, model training, and evaluation.

### 1.1 Data Ingestion and Initial Preparation

The process begins with a *CSV Reader*, which imports the dataset *imdb* containing textual data and corresponding sentiment labels.

A *Table Manipulator* is then used to adjust the dataset structure, followed by a *Table Cropper* to retain only relevant columns for further processing.

### 1.2 Text Preprocessing

The textual data undergoes several preprocessing steps:

- **String Cleaner:** removes noise such as punctuation and special characters.
- **Strings to Document:** converts raw text into KNIME document format.
- **POS Tagger:** assigns part-of-speech tags.
- **Stanford Lemmatizer:** reduces words to their base form.

After preprocessing, the *Document Data Extractor* retrieves structured textual data.

### 1.3 Feature Engineering

- **Bag of Words Creator:** generates term frequency features.
- **Document Vector:** transforms text into numerical vectors.

## 1.4 Data Preparation for Modeling

- **Table Updater:** merges generated features with labels.
- **Table Cropper:** keeps only relevant columns.
- **Table Partitioner:** splits the dataset into training and testing sets.

## 1.5 Model Training and Prediction

- **SVM Learner:** trains a Support Vector Machine model.
- **SVM Predictor:** generates predictions on the test set.

## 1.6 Evaluation and Visualization

- **Scorer:** computes evaluation metrics.
- **ROC Curve:** evaluates classification performance.

# 2 Decisions Made

- **Linguistic preprocessing:** POS tagging and lemmatization were used to improve normalization.
- **Bag of Words representation:** chosen for simplicity and interpretability.
- **Use of SVM:** selected due to its effectiveness in high-dimensional spaces.
- **Feature integration:** ensured correct alignment between features and labels.
- **Train-test split:** adopted for evaluation on unseen data.
- **ROC analysis:** included to complement traditional metrics.

# 3 Results

The model performance was evaluated using the Scorer node.

## 3.1 Class 1

- True Positives: 69
- False Positives: 38

- True Negatives: 72
- False Negatives: 46
- Recall (Sensitivity): 0.600
- Precision: 0.645
- Specificity: 0.655
- F1-score: 0.622

### **3.2 Class 0**

- True Positives: 72
- False Positives: 46
- True Negatives: 69
- False Negatives: 38
- Recall (Sensitivity): 0.655
- Precision: 0.610
- Specificity: 0.600
- F1-score: 0.632

### **3.3 Overall Performance**

- Overall F1-score: 0.62

### **3.4 ROC Curve Analysis**

The model achieved an Area Under the Curve (AUC) of 0.649, in figure 1, indicating performance above random classification but with moderate discriminative ability.

## **4 Critical Analysis**

### **4.1 Moderate Performance**

The model shows moderate effectiveness, with F1-scores around 0.62–0.63 and AUC of 0.649.

## 4.2 Class Behavior

The classifier presents a trade-off between recall and precision across classes, indicating slightly unbalanced predictive behavior.

## 4.3 Limitations of Bag of Words

The Bag of Words approach ignores word order and semantic context, limiting its ability to capture complex linguistic patterns.

## 4.4 Evaluation Strategy

The use of a single train-test split may limit the robustness of performance estimation.

# 5 Conclusion

The KNIME workflow successfully implements a structured pipeline for binary sentiment classification, integrating preprocessing, feature extraction, and machine learning.

The model achieved an F1-score of approximately 0.62 and an AUC of 0.649, demonstrating moderate performance.

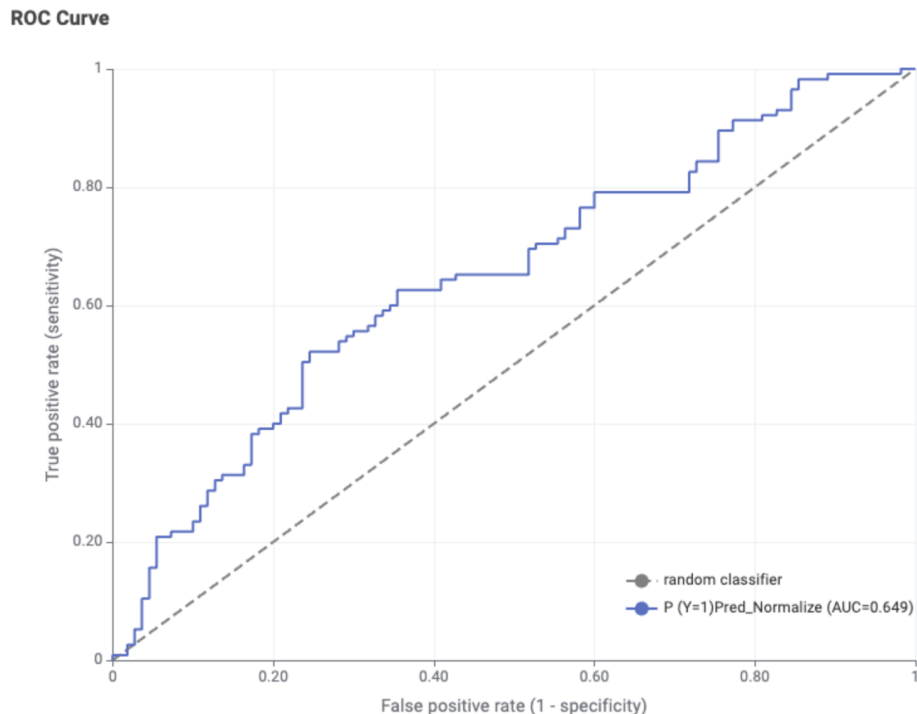


Figure 1: A Receiver Operating Characteristic (ROC) curve is a graphical plot illustrating the performance of a binary classifier across all thresholds, mapping true positive rate (Sensitivity) against false positive rate.

## 5.1 Github

The repository of project in <https://github.com/diasluciana/Knime1.git>

## 5.2 Future Improvements

Future work could focus on:

- Hyperparameter testing and optimization for the SVM model;
- Application of statistical evaluation methods (e.g., t-tests) to compare model performance;
- Use of class balancing techniques to improve robustness and predictive performance.